```
public class BookingInputModel
{
    public Property Property { get; set; }
    [Display(Name = "No. of guests")]
    public int NumberOfGuests { get; set; }
    [DataType(DataType.Date), Display(Name = "Arrival")]
    public DateTime? StartDate { get; set; }
    [DataType(DataType.Date), Display(Name = "Departure")]
    public DateTime? EndDate { get; set; }
}
```

In addition to `Property`, this class includes properties for the form fields you will add to the modal. The `BookingInputModel` will replace `Property` as the model for the modal. You haven't included any `BindProperty` attributes because you will pass `BookingInputModel` as a parameter to the handler method that processes the request. The code for the handler method follows in listing 11.9. It calculates the total cost of a stay, based on the number of days, the day rate, and the number of guests. This is also added to the `CityModel`.

Listing 11.9 The `OnPostBooking` handler method

```
public ContentResult OnPostBooking(BookingInputModel model)
{
    Var numberOfDays = (int)(model.EndDate.Value -
    ➥ model.StartDate.Value).TotalDays;
    var totalCost = numberOfDays * model.Property.DayRate *
    ➥ model.NumberOfGuests;
    return Content(totalCost.ToString("c"));
}
```

This handler method returns a `ContentResult`, which is a way of returning a string as a response. It is not something you are likely to use very often in real-world apps. However, it helps simplify this example. In addition, the fundamental logic for calculating the cost would usually belong in a service in a business logic layer. But again, I want to keep the example simple. The next listing shows the form added to the